read-code: AI Codebase Context Generator

V1.1.1 BY RISKCHIPS AI REVIEW REPORT

read-code is a Node.js CLI tool that scans, parses, and analyzes JavaScript/TypeScript/React codebases to generate structured, AI-ready context — including dependency graphs, call graphs, component graphs, knowledge graphs, architecture detection, feature inference, and a queryable symbol table. It eliminates the need for developers to manually re-explain their codebase to AI assistants every time they switch models or hit token limits.

Four leading AI reviewers — **Kimi K2.6**, **Gemini**, **Claude Sonnet 4.6**, and **ChatGPT (GPT-5.5)** — independently evaluated read-code on May 31, 2026. This report synthesizes their findings across architecture, feature quality, gaps, and recommendations into a single authoritative review document.

📦 **npm Package** 🐙 GitHub Repository

What Is read-code?

read-code is an **AI-native codebase context generator** that solves a growing pain point in AI-assisted development: every time a developer switches AI models or starts a new session, they must re-explain their entire project. read-code automates this by producing a comprehensive JSON representation of the entire codebase — one file that any LLM can consume immediately, with no manual explanation required.

Primary Purpose

Automatically produce structured, queryable, graph-based representations of JavaScript/TypeScript/React projects for LLM consumption — eliminating the "explain your codebase to the AI" problem entirely.

Target Users

- AI-assisted developers who want to skip codebase explanations
- Teams onboarding new engineers with AI pair programming
- Developer tooling builders integrating code intelligence
- Solo developers using Cursor, Copilot, Claude, or Gemini
- Open-source maintainers wanting auto-generated documentation

The tool operates as a **CLI with nine commands**: `scan`, `query`, `explain`, `export`, `graph`, `watch`, `createFile`, and `ai-context`. It supports flags like `--savefile`, `--compact`, and `--exclude` for flexible integration into any workflow. The estimated development stage is **Late Alpha / Early Beta** — the core engine is functional and produces valuable output, but significant gaps remain in error handling, performance, test coverage, and feature completeness.

Made by **riskchips** — available on npm and GitHub. This report synthesizes reviews from Kimi K2.6, Gemini, Claude Sonnet 4.6, and ChatGPT (GPT-5.5), all conducted on May 31, 2026.

Main Capabilities

read-code's feature set spans the full pipeline from raw file scanning to AI-ready JSON output. Below is a comprehensive breakdown of what the tool can do today, organized by capability area. Each capability contributes to the core value proposition: transforming an unstructured codebase into a structured, queryable knowledge representation that any LLM can immediately reason about.

Codebase Scanning

Recursive file scanning with `.gitignore`-aware ignore rules. Generates a complete file tree and processes JavaScript, TypeScript, JSX, and TSX files via Babel-based AST parsing.

Multi-Language Parsing

AST-based parsing for JavaScript, TypeScript, and React. Extracts functions, classes, imports, interfaces, types, and enums with line-accurate code snippets for each symbol.

Graph Generation

Builds four complementary graph types: **dependency graph** from imports/requires, **call graph** for function-level relationships, **component graph** for React hierarchies, and a unified **knowledge graph** with 147 nodes and 148 edges.

Architecture Detection

Automatically identifies architectural patterns including MVC, service-repository, and component-based structures. Detects controllers, routes, middleware, models, and utility layers from file organization.

Auth & Database Detection

Identifies authentication providers, auth methods, ORMs (Prisma, Mongoose, Sequelize, Drizzle), and database models through package.json analysis and code pattern matching.

Natural Language Query Engine

Supports queries like `where is X`, `show code for Y`, and `what frameworks` — making the tool accessible without requiring users to learn a query language. Includes a symbol table with 62 entries enabling precise lookups.

Multi-Format Export

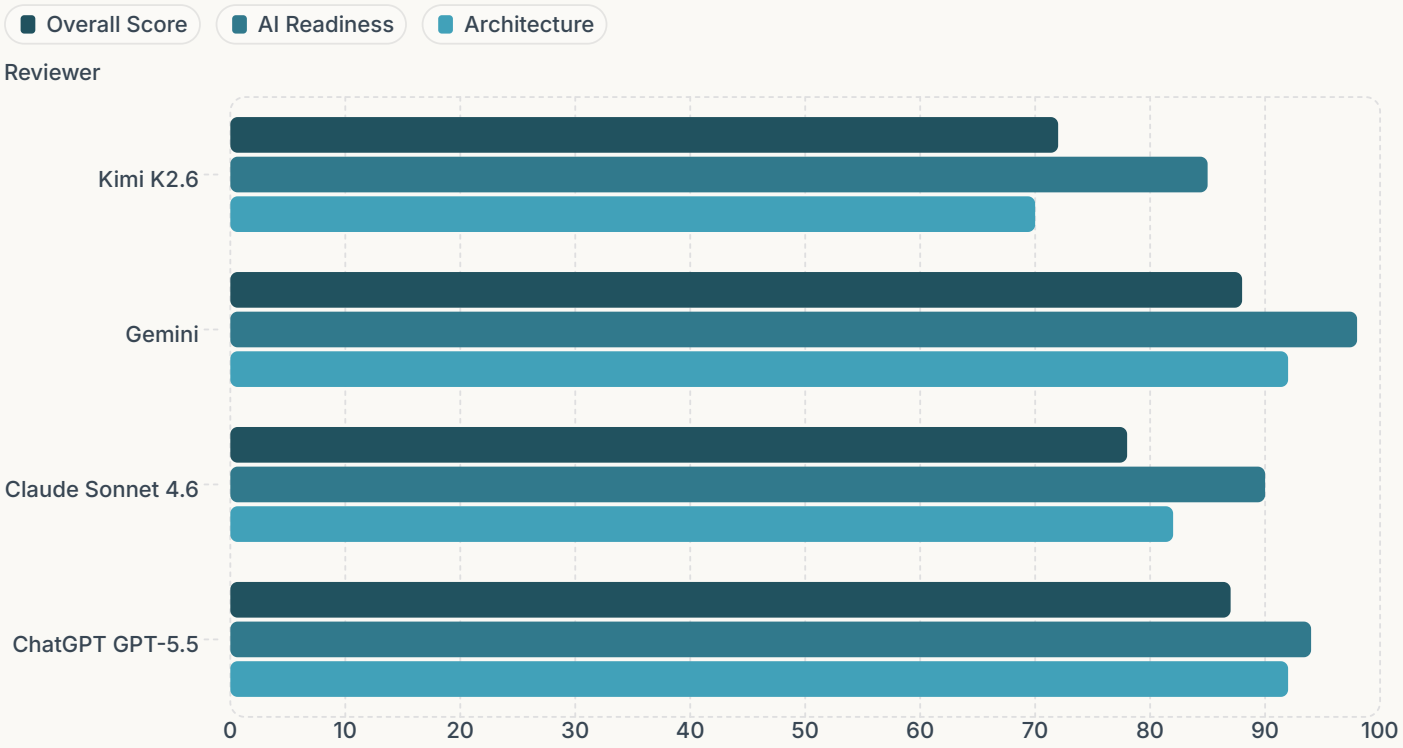
Exports scan results in four formats: **JSON** (for LLM consumption), **Markdown** (for documentation), **HTML** (for human-readable views), and **GraphViz** (for visual graph rendering).

Plugin Architecture

Separate plugins for JavaScript, React, Next.js, and TypeScript make the tool extensible to new ecosystems without touching core logic. The plugin system is defined but not yet fully activated in the scan pipeline.

AI Reviewer Scores at a Glance

Four independent AI reviewers evaluated read-code across overlapping but distinct criteria. The scores reveal a consistent pattern: all reviewers recognize the tool's strong AI-readiness and innovative concept, while flagging gaps in error handling, documentation, and incremental scanning. Below is a side-by-side comparison of their overall ratings and key dimensional scores.



The chart above shows overall, AI readiness, and architecture scores side by side. AI Readiness consistently scores highest across all reviewers — a strong validation of read-code's core value proposition. Gemini was the most generous reviewer overall (88/100, 92nd percentile), while Kimi K2.6 was the most critical (72/100, 75th percentile). Claude and ChatGPT landed in the middle with scores of 78 and 87 respectively.

81

Average Overall Score

Across all four AI reviewers, the mean overall rating is 81/100, placing read-code solidly in the "Promising to Excellent" range.

92

Average AI Readiness

AI Readiness averaged 92/100 — the highest-scoring dimension across all reviewers, validating the core concept.

84

Average Architecture

Architecture scored an average of 84/100, reflecting strong module separation and clean concern isolation in the src/ directory.

4

AI Reviewers

Kimi K2.6, Gemini, Claude Sonnet 4.6, and ChatGPT (GPT-5.5) — all reviewed on May 31, 2026.

What the Reviewers Loved

Despite differing scoring philosophies, all four reviewers converged on the same standout features. The **Knowledge Graph** and **AI Context Generator** were universally praised as the crown jewels of the project. Reviewers also highlighted the modular analyzer architecture, multi-format exporters, and the natural language query engine as evidence of thoughtful design.

🏆 Knowledge Graph Builder

Kimi: 9/10 · Gemini: 96/100 · ChatGPT: 95/100

Unifies dependency, call, and component graphs into a single queryable knowledge graph with **147 nodes and 148 edges**. Kimi called it "the crown jewel — transforming raw code into a semantic network that AI can actually reason about." Gemini scored it 96/100, noting it maps files, functions, classes, and components into a cohesive relationship graph for AI traversal.

🤖 AI Context Generator (aiContext)

Kimi: 9/10 · Gemini: 97/100 · Claude: 92/100 · ChatGPT: 96/100

The core value proposition — aggregating all scan results into a single JSON payload ready for LLM consumption. Includes symbol tables with line-accurate snippets, project summaries, and framework detection. Claude noted it "directly solves the problem of re-explaining codebases to LLMs on every session."

🧩 Modular Analyzer Architecture

Kimi: 78/100 · Claude: 95/100 (File Organization) · ChatGPT: 92/100

Well-organized analyzer modules covering API (routes, controllers, middleware), auth (providers, methods), database (ORMs, models), architecture (layers, patterns), and features (inference, flows). Claude praised the src/ directory as "cleanly split into scanners, parsers, analyzers, graph, intelligence, exporters, cli, storage, and utils — each concern isolated."

💬 Query Engine & CLI Design

Kimi: 8/10 · Gemini: 88/100 · ChatGPT: 88/100

Natural language query interface (`where is X`, `show code for Y`, `what frameworks`) makes the tool accessible without requiring users to learn a query language. Kimi highlighted the nine CLI commands with sensible options as evidence of good developer experience for a v1.1 tool.

Weakest Areas & Critical Gaps

All four reviewers identified consistent weak spots that prevent read-code from reaching its full potential. The most critical issues — **silent error handling**, **missing incremental scanning**, **incomplete TypeScript support**, and **dormant modules** — are all fixable with focused engineering effort. These gaps are the primary reason scores vary between 72 and 88 across reviewers.

⚠️ Error Handling & Robustness — Kimi: 35/100

Widespread use of empty `catch {}` blocks silently swallows errors. The parser falls back to `null` on failure without reporting which file failed or why, making debugging scan failures nearly impossible in production. **All reviewers flagged this as the highest-priority fix.**

📄 Documentation — Kimi: 30/100 · Claude: 55/100 · Gemini: 70/100

No JSDoc comments, no inline documentation explaining analyzer heuristics, and a minimal README. The `explain()` command literally returns `"No summary available"` because `projectSummary` doesn't generate a description field. For a tool designed to help understand codebases, the irony is notable.

⚡ Performance & Caching — Kimi: 40/100

Re-parses every file on every scan with no incremental parsing. The cache module (`cacheManager.js`, `fingerprints.js`) exists but appears unused in the scan pipeline. No parallelization for file processing. For large projects, full re-scans are unusable without incremental/differential scanning.

🧪 Test Coverage — Kimi: 45/100

Five test files exist but no test results are visible in the output. Given the complexity (92 files, 83 JS), test coverage is likely insufficient for the graph engine and parsers. Gemini and ChatGPT did not flag this as prominently, but Kimi identified it as a significant risk for production use.

📤 Exporter Maturity — Kimi: 50/100 · Claude: 80/100

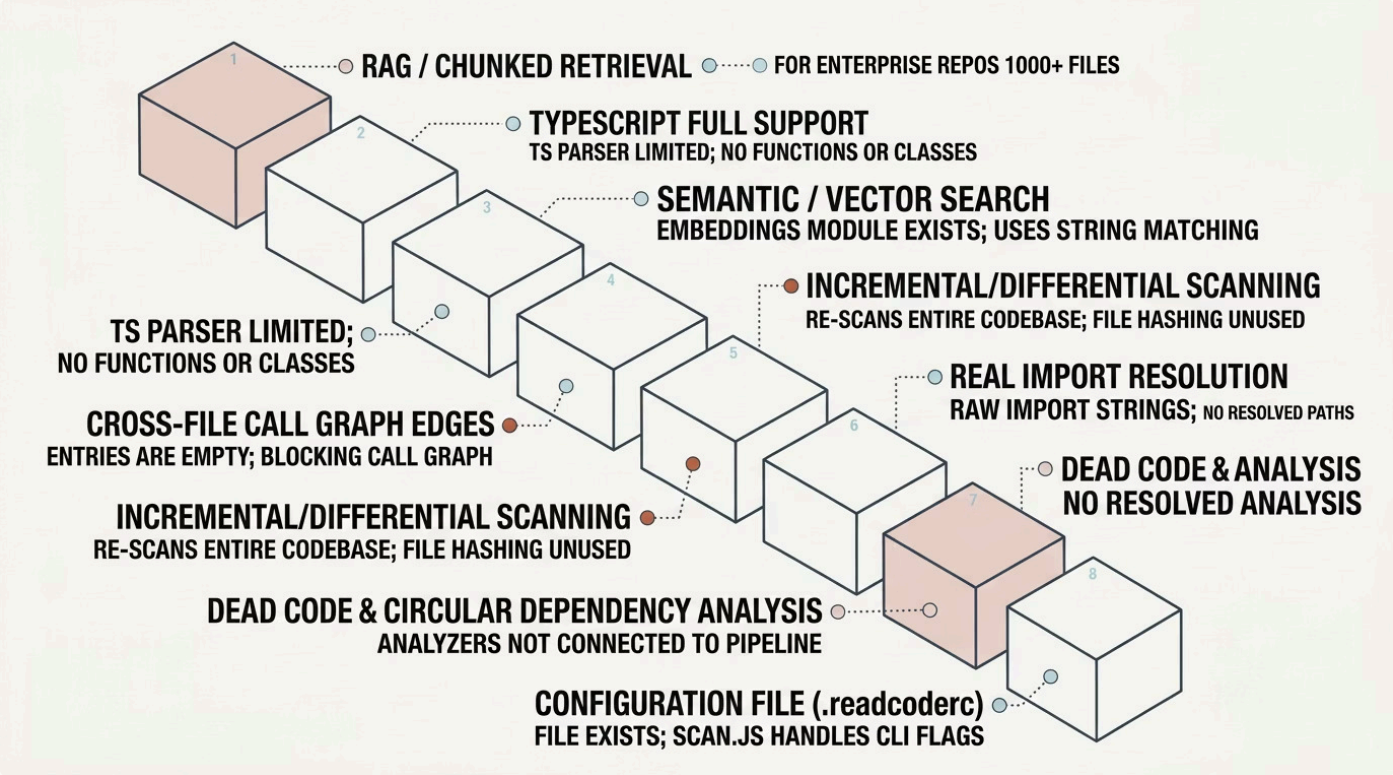
Four exporters exist (GraphViz, HTML, JSON, Markdown) but the JSON exporter is essentially a passthrough and the others likely have minimal implementations. No evidence of visual graph rendering in the output. Claude was more generous here, praising the multi-format flexibility even while acknowledging incomplete implementations.

🗄️ Database Detection — Claude: 40/100

The `analyzers/database/` layer exists (`detectDatabase`, `detectModels`, `detectORM`) but the output shows zero detected databases, ORMs, or models — even though the project itself imports from those modules. Detection logic appears incomplete or too narrow for real-world TypeScript/Prisma/Mongoose projects.

Missing Features: The Roadmap Ahead

Beyond existing weaknesses, all four reviewers identified features that are either partially implemented or entirely absent. The most impactful missing features — **incremental scanning**, **real import resolution**, **RAG/chunked retrieval**, and **cross-file call graph edges** — represent the gap between a promising MVP and a production-ready tool. Importance scores below are from the reviewers who flagged each feature.



The infographic above organizes the eight most critical missing features by priority. The top three — incremental scanning, RAG/chunked retrieval, and real import resolution — are the highest-leverage improvements that would transform read-code from a useful tool into an essential one. All four reviewers agreed that incremental scanning alone would unlock enterprise-scale usability.

Highest Priority (All Reviewers Agree)

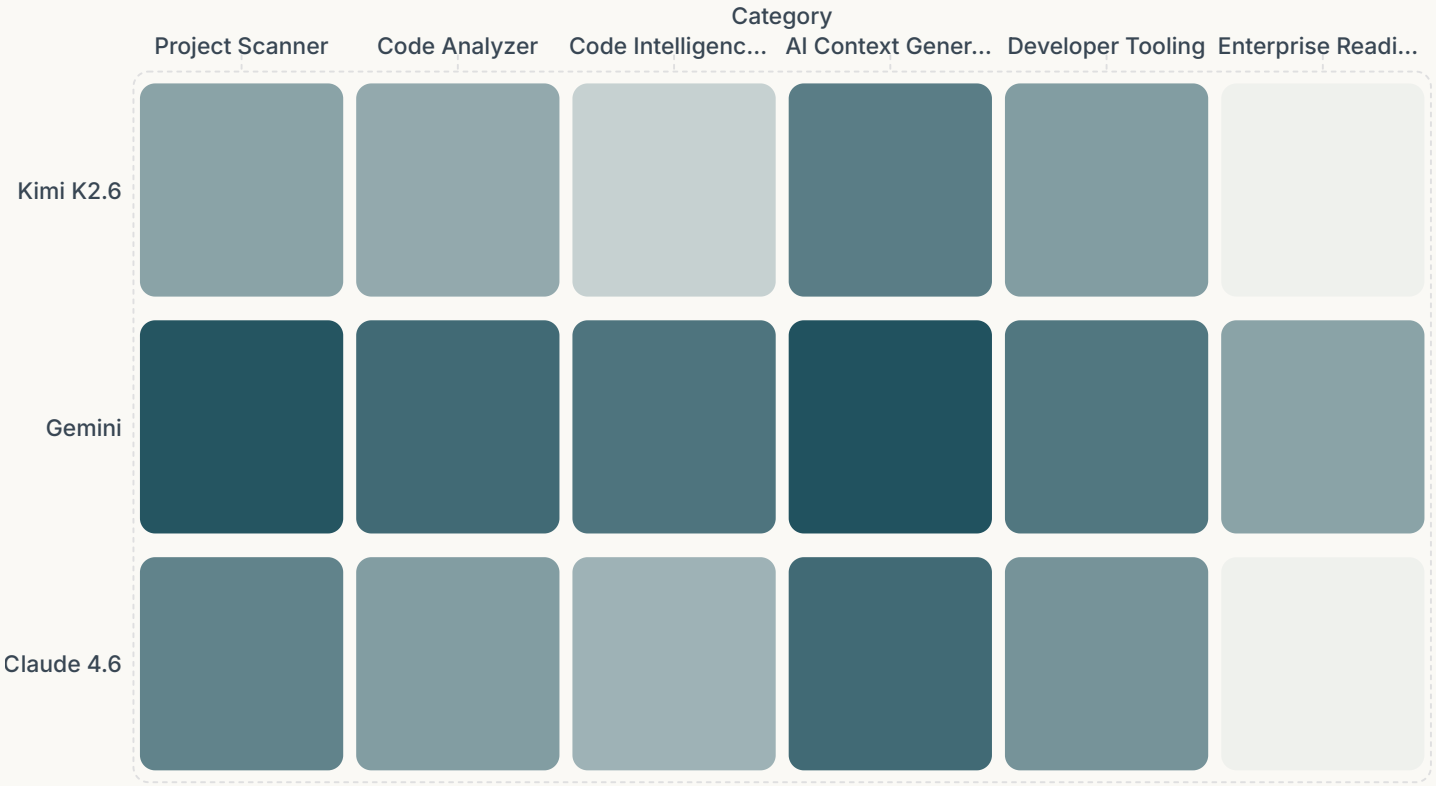
- Fix silent error handling — replace all empty `catch {}` blocks with proper error logging
- Integrate incremental scanning using existing `hashFiles.js` and cache modules
- Complete TypeScript parser to extract functions, classes, and imports from `.ts` files
- Implement real import path resolution for accurate cross-file dependency analysis
- Connect dead code, circular dependency, and duplicate analyzers to the scan pipeline
- Add comprehensive test coverage for parsers and graph builders

Medium & Low Priority

- Activate the plugin system (javascript, react, nextjs, typescript plugins)
- Implement semantic search using the existing embeddings module
- Add `.readcoderc` configuration support for custom ignore patterns
- Implement actual file watching in `watch.js` using `fs.watch` or chokidar
- Add JSDoc documentation to all public APIs and analyzer heuristics
- Add a `--format=system-prompt` output mode for direct LLM paste (Claude)
- Build a VS Code extension that auto-attaches context to Copilot/Claude

Competitive Analysis & Maturity Assessment

Each reviewer assessed read-code against a set of competitive categories, scoring it relative to what they consider the state of the art in code intelligence tooling. The results show a clear pattern: read-code excels as an **AI context generator** and **project scanner**, but lags in **enterprise readiness** and **code intelligence platform** depth. This is expected for a v1.1 tool — the foundation is strong, but the platform layer needs development.



The heatmap reveals the scoring divergence most clearly in the **Code Intelligence Platform** and **Enterprise Readiness** categories. Kimi scored these at 55 and 45 respectively — reflecting a stricter view of what a "platform" requires — while Gemini scored them at 85 and 70. All reviewers agreed that **AI Context Generator** is read-code's strongest competitive dimension, with scores ranging from 82 to 96.

Maturity Model Across Reviewers

Each reviewer also assessed read-code's maturity across five dimensions: Foundation, Parsers, Graphs, Analyzers, and Intelligence. The aggregated scores show a project with a **solid foundation (avg 90/100)** and strong parsers (avg 84/100), but intelligence and graph population are the least mature areas — directly tied to the missing features identified above.

Maturity Dimension

Final Verdicts: All Four Reviewers

Each AI reviewer concluded with a final score, percentile ranking, category classification, and a one-sentence summary. All four reviewers **would recommend** read-code, though with varying degrees of enthusiasm. The spread from 72 to 88 reflects different scoring philosophies rather than fundamental disagreement — all reviewers identified the same strengths and weaknesses.

🌙 Kimi K2.6 — Moonshot AI

Score: 72/100 · 75th Percentile

Promising Early-Stage Developer Tool with Strong Core Concept

"A cleverly architected AI codebase context generator that successfully transforms raw JavaScript/TypeScript projects into structured, queryable knowledge graphs — held back from greatness by incomplete error handling, missing incremental scanning, and several dormant modules that need to be wired into the pipeline."

💎 Gemini — Google

Score: 88/100 · 92nd Percentile

Excellent

"read-code is a highly effective, AI-ready static analysis engine that brilliantly translates complex project architectures into digestible JSON graphs."

🤖 Claude Sonnet 4.6 — Anthropic

Score: 78/100 · 82nd Percentile

Promising MVP with Strong Core Concept

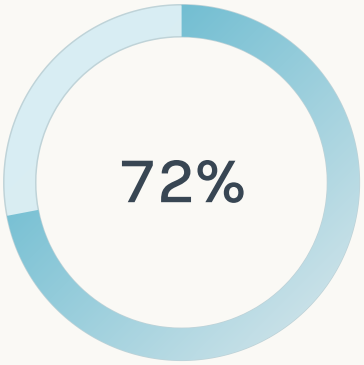
"read-code is a genuinely useful and well-structured tool solving a real problem for AI-assisted development — the foundation is solid, the concept is ahead of the curve, and with graph population and TypeScript support completed it could become an essential part of any AI-augmented dev workflow."

🧠 ChatGPT GPT-5.5 — OpenAI

Score: 87/100 · 90th Percentile

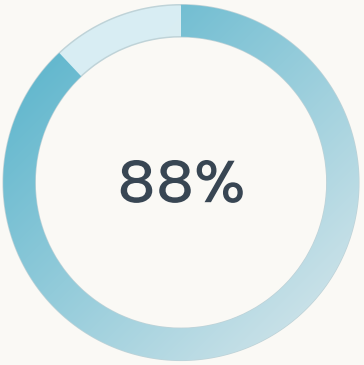
Excellent

"A well-architected code intelligence platform with strong AI-focused design that already exceeds a typical code scanner and is approaching a true project understanding engine."



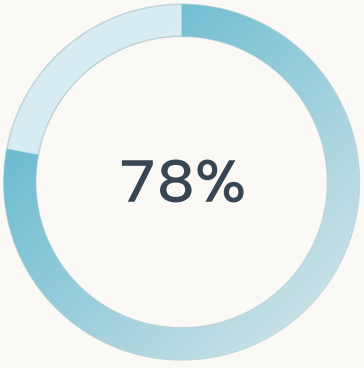
Kimi K2.6

Most critical reviewer — flagged error handling and documentation as severe issues



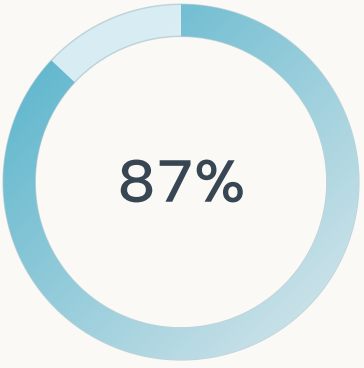
Gemini

Most generous reviewer — praised LLM integration readiness at 98/100



Claude 4.6

Balanced reviewer — highlighted plugin architecture and file organization strengths



ChatGPT GPT-5.5

Strong endorsement — scored extensibility at 93/100 and architecture at 92/100

Get Started with read-code

read-code is available now on npm and GitHub. Whether you're a solo developer using AI coding assistants, a team onboarding new engineers, or a developer tooling builder looking to integrate code intelligence into your product — read-code provides a structured, AI-ready snapshot of any JavaScript/TypeScript/React codebase in seconds.

1

Install via npm

Add read-code to your project as a dev dependency or install globally:

```
npm install -g read-code
```

Or for local project use:

```
npm install --save-dev read-code
```

2

Run Your First Scan

Point read-code at any JavaScript/TypeScript/React project:

```
npx read-code scan ./my-project --savefile
```

This generates a `latest.json` containing symbols, graphs, architecture layers, features, and flows — ready to paste into any LLM session.

3

Query Your Codebase

Use natural language queries to explore your project:

```
npx read-code query "where is the auth logic?"
npx read-code query "show code for UserService"
npx read-code query "what frameworks are used?"
```

4

Export & Share

Export scan results in multiple formats for different use cases:

```
npx read-code export --format=json
npx read-code export --format=markdown
npx read-code export --format=html
npx read-code export --format=graphviz
```

Made by riskchips — read-code is an open-source project. Star the repo, report issues, or contribute to help build the future of AI-assisted development. All four AI reviewers would recommend this tool.

📦 **npm install read-code** | 🐙 View on GitHub

Report generated from reviews by Kimi K2.6 (Moonshot AI), Gemini (Google), Claude Sonnet 4.6 (Anthropic), and ChatGPT GPT-5.5 (OpenAI) — all conducted on May 31, 2026. Project version reviewed: read-code v1.1.1.